

Project 2 Robot

Casey Stijlaart, 466488

Fontys University of Applied Science

21-01-2022

Table of Contents

Introduction	1
System Design	2
Hardware	2
Use Cases	2
System behavior	4
Sequence diagrams	4
State machine	5
Activity diagrams	6
Research	8
Feedback Control	8
Algorithm	8
Reflection	10
360 degrees servo	10
References	10

Introduction

Project 2 consists out making the FHICT bot move around without bumping into any obstacles. This will be done with the help of the attached sensors and a feedback control system. To be more specific, the Robot will be moving into a straight line until a object gets too close. When this occurs, the robot will making a random turn between -90 degrees and 90 degrees. The approach for getting towards the end result is described throughout this document.

System Design

Hardware

Nucleo F303RE

The chosen microcontroller has the possibilities to configure timers for the PWM signals.

Parallax Feedback 360° High-Speed Servo

The servo has the capability to rotate a full 360 degrees with the help of PWM signals. Based on the PWM signal, it can rotate clockwise and counter clockwise. Besides that, the servo has the ability to retrieve a feedback signal. This feedback signal can be used to control two servo's in the same manner. Regarding to this project, this function can be used to make the robot move in a straight line.

Ultrasonic sensor : HC-SR04

The ultrasonic sensor that measures distance by emitting a 8 bursts of 40kHz ultrasonic wave. When it encounters an object, the ultrasonic will reflect and return to the ultrasonic sensor. Based on how long the echo pin (the receiver) stays high, the distance is calculated.

Use Cases

In this section the interaction between the actors and system are described with the help of the use cases. Regarding to the actors, the ultrasonic sensor is seen as an actor as well due to roll it has in the complete system. In the tables [Table 1](#) and [Table 1](#) the use cases are described. The use case diagram is displayed in [Figure 1](#).

Description

Table 1. UC_001

UC_001	Move in straight line.
Description	The robot moves in a straight line.
Actor	Robot

Table 2. UC_002

UC_002	Turn random direction.
Description	When an object comes within 10cm, the robot makes a random turn in degrees.
Actor	Ultrasonic Sensor, robot

Diagram

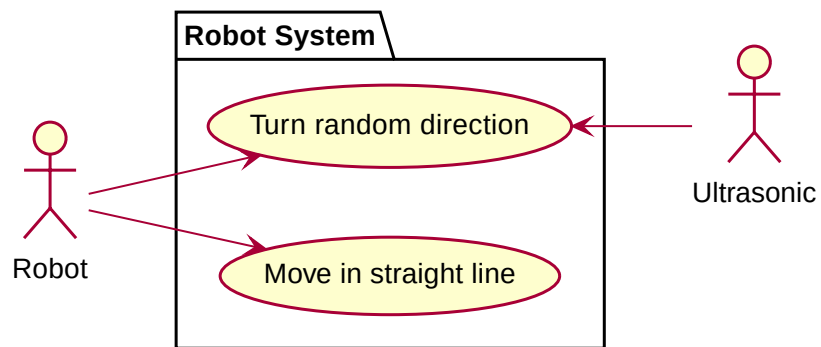


Figure 1. Use case diagram

System behavior

Sequence diagrams

Throughout this section multiple scenario's are described on how the system will be interacting with the the components. These scenario's are based on the use cases and are given in sequence diagrams. The modules that have previously been described are used as objects throughout the diagrams.

UC_001 : Move in straight line

Scenario 1

The robot has two initialized wheels and requests a general function, which individually calls each wheel to rotate on the given speed. The sequence diagram of this scenario is given in [Figure 2](#).

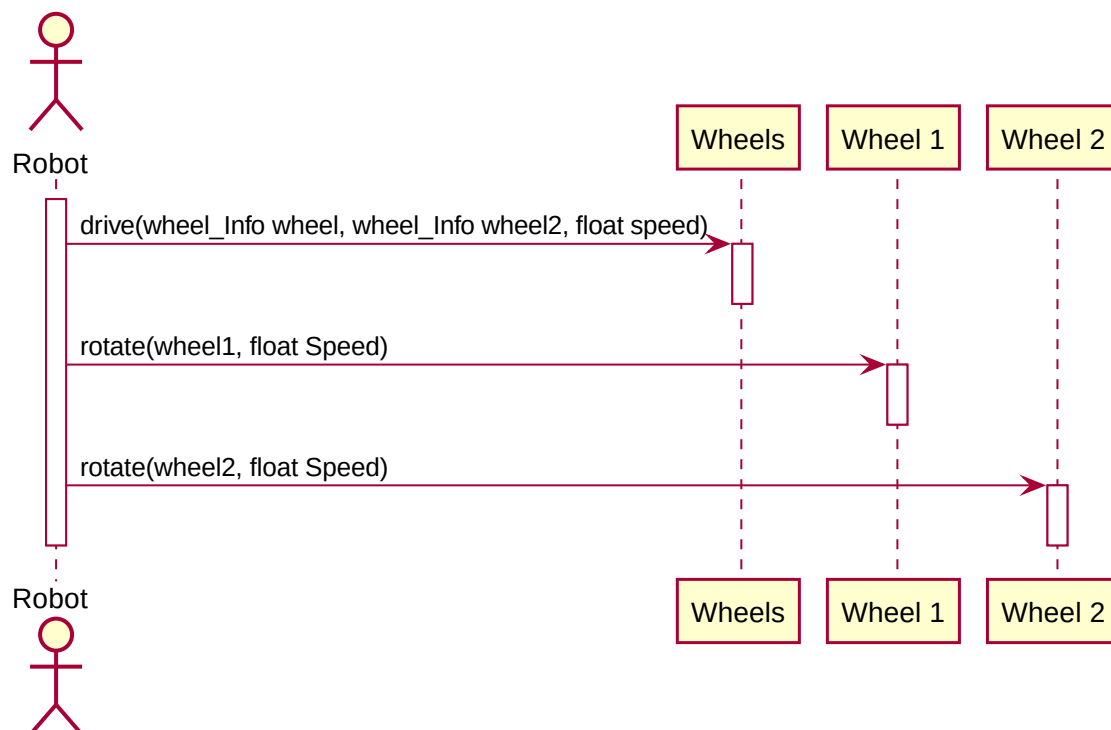


Figure 2. the robot moves in a straight line

UC_002 : Turn random direction

Scenario 1

The ultrasonic sensor measures that an object is within a 10cm range from the robot. Therefore the robot will randomly turn in a direction. [Figure 3](#).

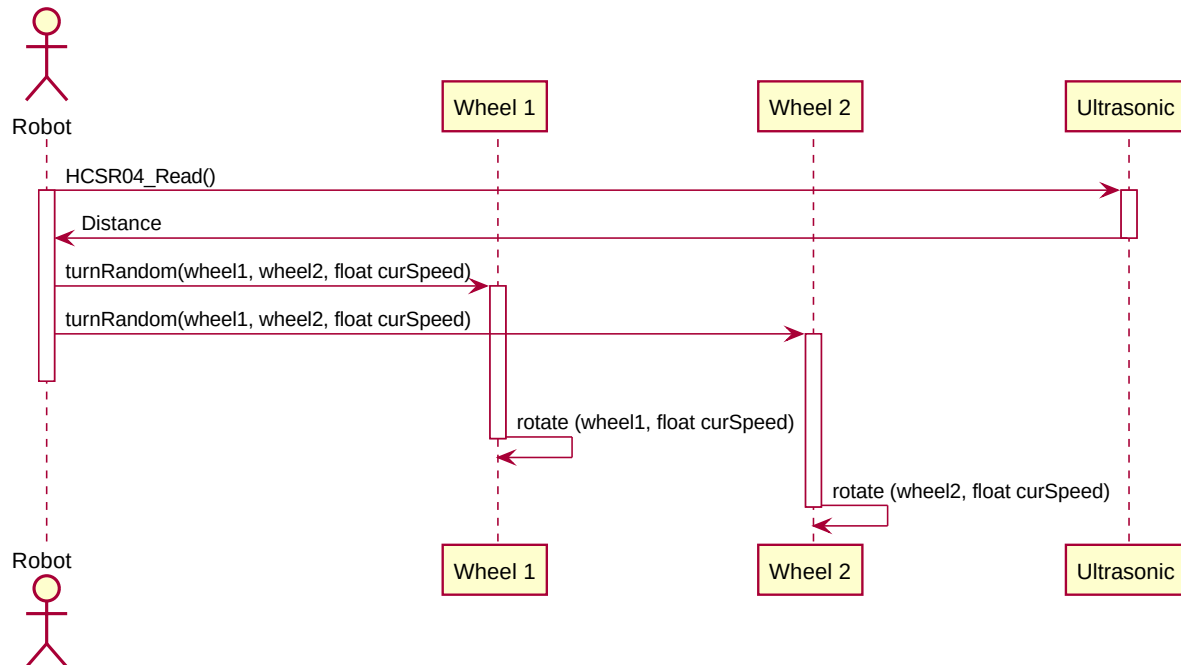
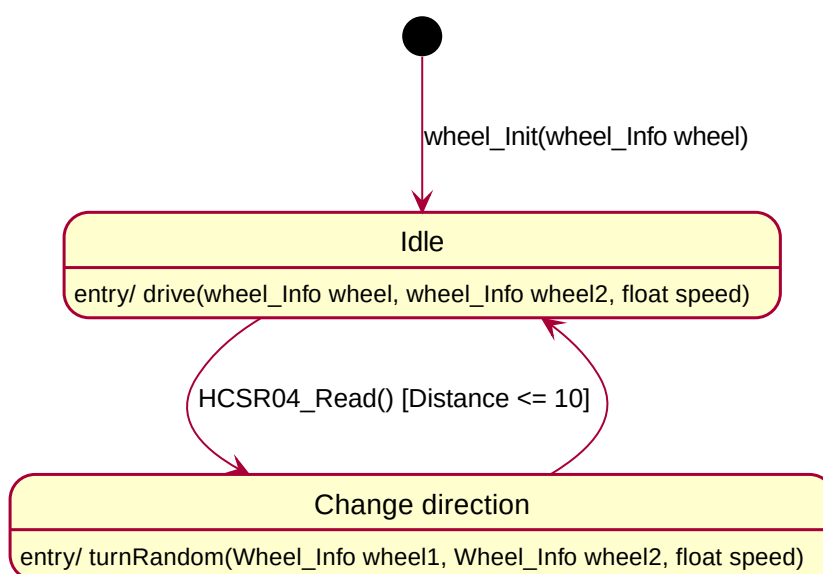


Figure 3. Turn random direction

State machine

The robot has two possible states, idle and turn in a random direction. The transition between these states are based on specific events which are triggered by for example the Ultrasonic sensor.



Activity diagrams

Each part of the robot has its own way on how it needs to perform to result in the wished for end result. To get an clear overview on how each component has to work, activity diagrams are created. These diagrams are used to help implementation to go smoother and to have an overall more clear view of each component.

Complete System

The complete system of the robot exists out of multiple components, each with their own purpose. See [Figure 4](#) for the activity diagram.

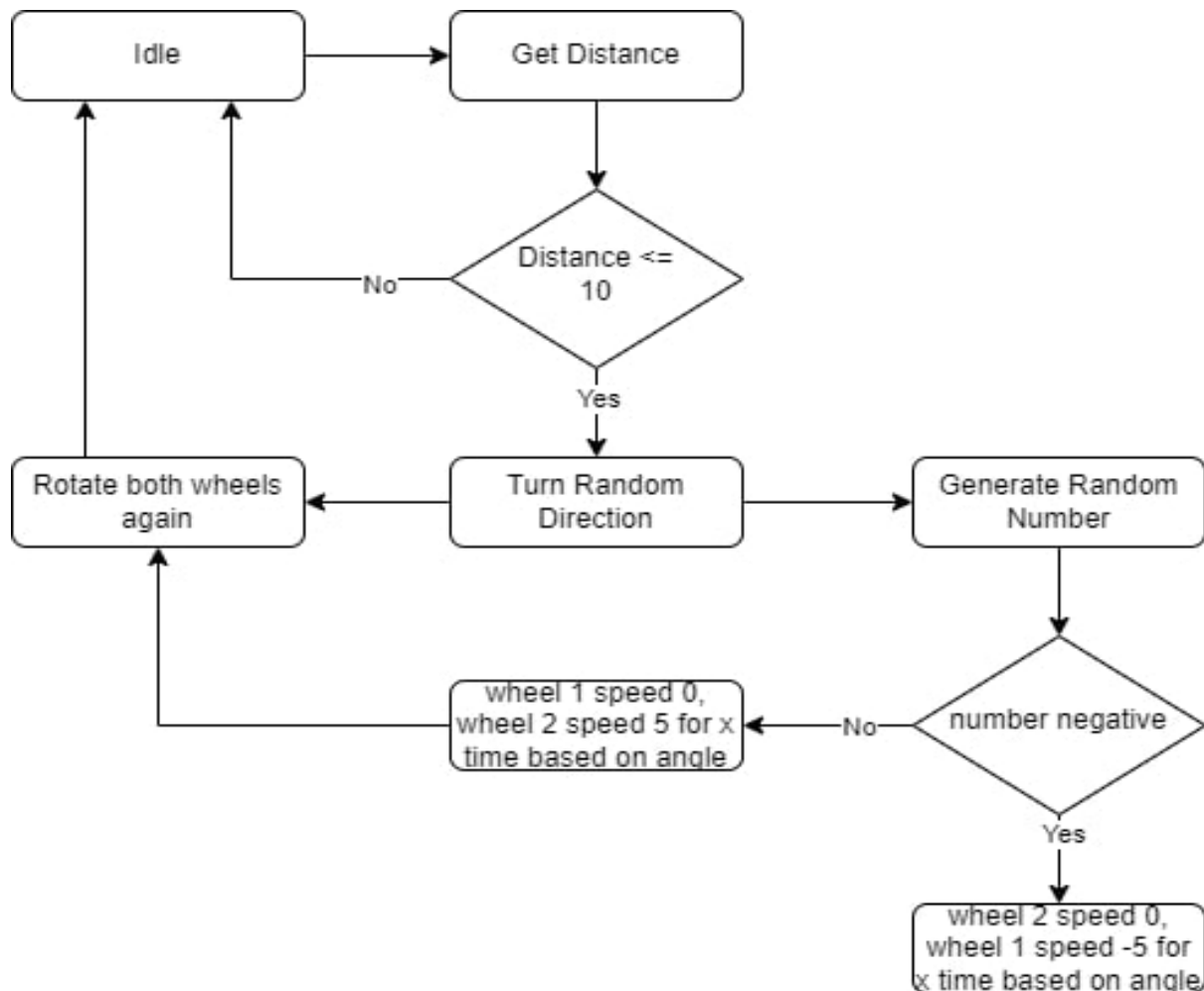


Figure 4. Complete System Activity Diagram

Feedback Control

The robot makes use of an feedback control system to control both of the servo's, making sure they are rotating at the same speed. To control these servo's, an interrupt is called every period and will check the current CCR values and based on these make the decision if action needs to be taken. See [Figure 5](#) for the activity diagram.

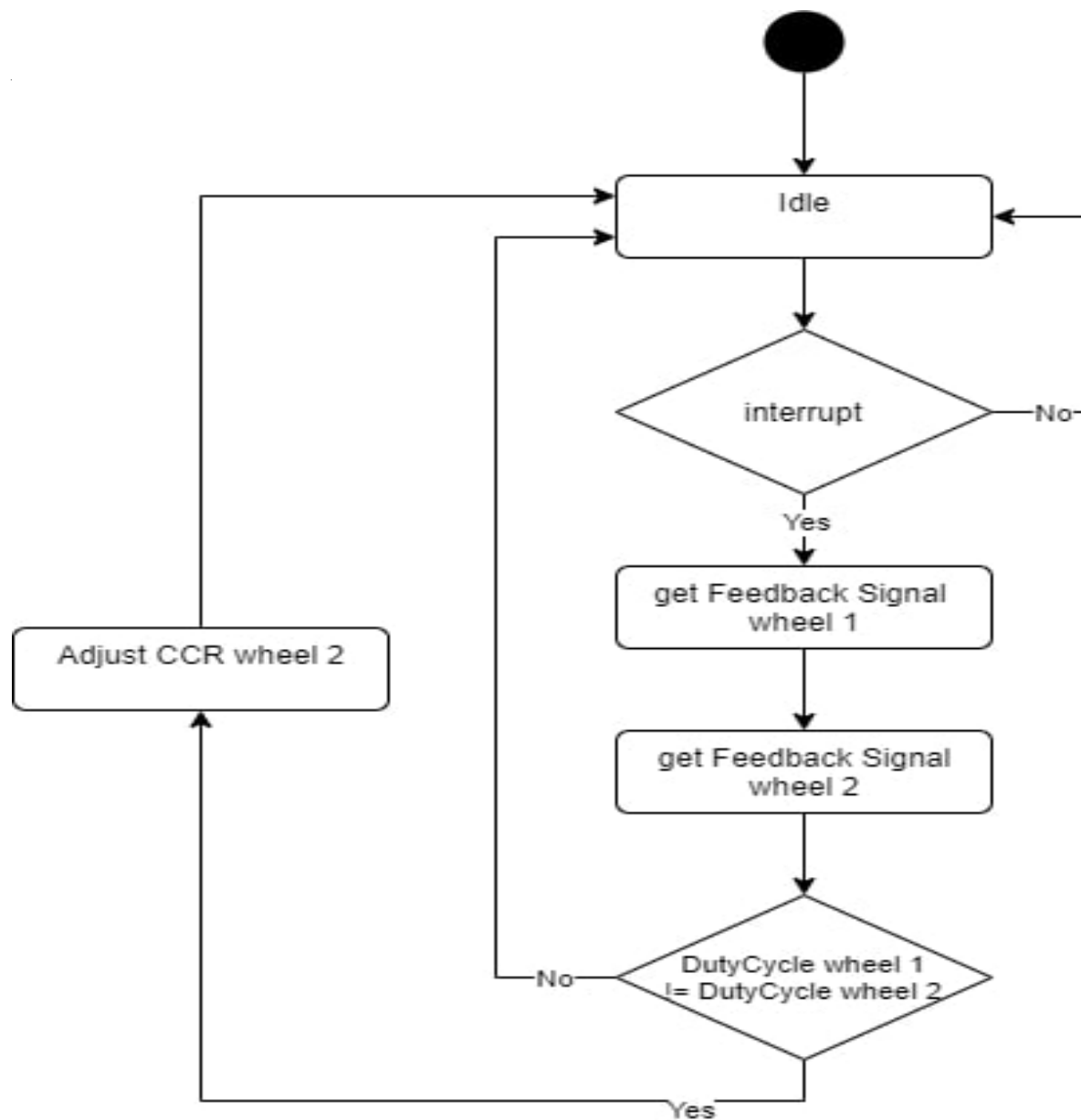


Figure 5. Feedback Control Activity Diagram

Research

Feedback Control

The feedback control is used to control the rotation of the servo's based reading the current input PWM signal of one of the servo's. To go into more detail, each servo has an PWM input signal. This signal is used to read the current position of the servo. To read this signal, the signal is measured and converted into a readable duty cycle period. With the help of converting it to an duty cycle period, the PWM output signal from each servo is compared and based on this action will be taken. The action that will be taken, is to adjust the PWM output of one servo based on the PWM input signal from both servo's. For example, wheel 1 has a duty cycle period from 0.074 and wheel 2 from 0.070. Therefor a newly calculated CCR value, for wheel 2, is calculated based on the goal duty cycle, which is the same as wheel 1. This value will be set as the new CCR value for the second wheel instance.

Algorithm

Parallax Feedback 360° High-Speed Servo

Speed

The control signal of the servo must be sent every 20ms. To achieve this the reload value of the used timer has to be calculated based on the expected period.

$$ARR = \text{period} * \text{clock speed} - 1$$

$$ARR = 0.02s * 16 * 10^6 - 1 = 320\ 000 - 1$$

The control signal can be manipulated in such a way that the RPM of the servo can be changed. The RPM helps with finding the minimum and the maximum speed of the servo in cm/s. Since the RPM itself in rad/s, it has to be converted into cm/s. To do so, the following calculations have to be made. Since one of the servo's is placed reversed, the speed for this one will be reversed.

$$v = \text{RPM} / 60 * (\text{diameter} * \pi)$$

$$v_{\text{min}} = 0 / 60 * (7\text{cm} * \pi) = 0 \text{ cm/s}$$

$$v_{\text{max}} = 140 / 60 * (7\text{cm} * \pi) = 51,31 \text{ cm/s}$$

To make the servo rotate based on these speeds, the CRR values have to be calculated. This is done with the help of the given tcontrol, the tcontrol is given in microseconds. $CRR = \text{Duty Cycle} * ARR$

Clockwise

$$CCR_{\text{min}} = 0.00148s / 0.02s * (320\ 000 - 1) = 23\ 680$$

$$CCR_{\text{max}} = 0.00128s / 0.02s * (320\ 000 - 1) = 20\ 480$$

Counter clockwise

$$CCR_{\text{min}} = (1520 * 10^{(-3)} \text{ ms}) / 20\text{ms} * (320\ 000 - 1) = 24\ 320$$

$$\text{CRR_max} = (1720 * 10^{(-3)} \text{ ms}) / 20\text{ms} * (320\,000 - 1) = 27\,520$$

Feedback Control

The timers for the servo's PWM input need be adjusted regarding to its PSC, so it results into an 1,1ms(1MHz) period. This is done by using the following formula.

$$\text{PSC} = \text{Timer Frequency} / 1\,000\,000 - 1$$

$$\text{PSC} = 16\,000\,000 / 1\,000\,000 - 1 = 16 - 1$$

For getting the current duty cycle of a wheel, the retrieved measurement is used to calculate it. Within this formula, the dutyscale is 1000, unitsFC is 360, dcMin is 29 and dcMax is 971.

$$\text{Duty Cycle} = (\text{dutyScale} * \text{tHigh}) / (\text{tHigh} + \text{tLow})$$

Ultrasonic sensor : HC-SR04

The duration of the pulse that is being returned to the sensor is measured in microSeconds. To convert this into the measured distance the following algorithm is applied. In this formula the time is in microseconds.

$$\text{Distance in CM} = \text{time} * 0.034 / 2$$

Reflection

360 degrees servo

When programming the servo a slight offset in the the tControl. These values did fall into the range that has been mentioned in the Parallax Feedback 360° High-Speed Servo datasheet. Besides that, as mentioned, one of the servo's has been placed backwards. This results in an opposite result. To resolve this the speed had to be negative when wanting to go forward and positive when going backwards. The found tControl values in microseconds and the found tControl per speed in cm/s can be seen in the given tables.

Table 3. tControl values in microseconds

	Forward max	Forward min	Backwards min	Backwards max
Wheel 1	1730	1530	1473	1255
Wheel 2	1730	1530	1473	1255

Table 4. tControl values in speed (cm/s)

	Forward max	Forward min	Backwards min	Backwards max
Wheel 1	51.31	0	0	-51.31
Wheel 2	-51.31	0	0	51.31

References

1. Frenken-Liskova, R. (2021). Robot Swarm Project. From Git: https://git.fhict.nl/technology/t-sem3-cb/-/blob/master/project/2_robot_swarm/Robot%20Swarm%20Project.docx
2. Parallax. (2017). 360 HS Servo. From Git: https://git.fhict.nl/technology/t-sem3-cb/-/blob/master/project/2_robot_swarm/Toolbox/900-00360-Feedback-360-HS-Servo-v1.1.pdf
3. Cytron Technologies (2013). HC-SR04 User Manual. From Git: https://git.fhict.nl/technology/t-sem3-cb/-/blob/master/project/2_robot_swarm/Toolbox/HCSR04_user_manual_1.pdf
4. HC-SR04 User Guide. From Git: https://git.fhict.nl/technology/t-sem3-cb/-/blob/master/project/2_robot_swarm/Toolbox/hc-sr04_ultrasonic_module_user_guidejohn.pdf
5. PWM INPUT in STM32. From Controller Tech: <https://controllerstech.com/pwm-input-in-stm32/>
6. Feedback 360° High Speed Servo Propeller C Example Code. From Parallax : <https://www1.parallax.com/downloads/feedback-360-high-speed-servo-propeller-c-example-code>